

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ace7351e-c4e\agent-a8a96f2231f9a6a78.jsonl`
- SHA-256: `b26536fabdd116b289b775bbe815b8485e17d139fed04ea820ef3f9e3c1fe0fe`
- Source modified: `2026-07-06T22:21:39+00:00`
- Imported at: `2026-07-08T16:00:07+00:00`
- project: `wf\_ace7351e-c4e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T22:19:43.195Z)

Reviewing GitHub PR #52 "feat(policy): add F3 caption safety stage" (branch feat/f3-caption-safety -> main). ADR 0012 F3.
+299/-41, 4 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr52 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f3-caption-safety:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F3 DOES:

- policy.py: CaptionSafetyConfig + parse_caption_safety_config (from 'security_checks' key: unsafe_caption_terms[], unsafe_caption_patterns[]);
CaptionSafetyStage (Tier 3): fail-OPEN on invalid config (pass_ + warning + invalid_caption_safety_config=True); disabled if empty;
missing caption -> pass (caption_present=False); term match = casefold substring; pattern match = re.search(pattern, caption) (case-SENSITIVE
unless user adds (?i)); regexes validated via re.compile at parse time (invalid regex -> ValueError -> fail-open).
- telethon_client.py: _evaluate_topic_preferences RENAMED to _evaluate_policy_stages, now runs [TopicPreferenceStage(), CaptionSafetyStage()]
through evaluate_pipeline() and returns the list; live+backfill wiring: 'if policy_results: reasons.extend(...); matched = policy_results[-1].passed'.
- Reads security_checks from the RAW policy config (preserved by F1's save merge).
on_new_message/backfill_user are '# pragma: no cover'.

CONFIRMED by the main reviewer: gate green (523 passed @ 95.77%); ReDoS is REAL ? the valid pattern '(a+)+\$' on a 40-char caption hangs
re.search past a 6s timeout (rc=124), blocking the synchronous event loop. Feature is dormant today (no command sets security_checks).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario, ranked.
Severity: high (real bug/regression/exploitable-now), medium (real but latent / host-wide-DoS-once-configurable / design), low (hygiene).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr52/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /
pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity (recall: dormant-but-real host-DoS = medium, hygiene = low).
FINDING (security): User-supplied caption regexes run via re.search on the shared asyncio event loop with no timeout or complexity bound (ReDoS -> host-wide ingestion stall)
severity medium | src/tg_video_filter/policy.py:294

summary: CaptionSafetyStage.evaluate calls re.search(pattern, caption) for each configured unsafe_caption_pattern with no timeout, length cap, or complexity limit; a valid-but-pathological pattern plus an attacker-controlled caption causes catastrophic backtracking that blocks the single shared manager event loop, starving ingestion for all users on the host.

scenario: security_checks.unsafe_caption_patterns is set to a valid regex like '(a+)\$' (compiles cleanly, so the re.compile validation in parse_caption_safety_config passes). An attacker posts a video in any watched channel with a ~29-char caption 'aaaa...a!'. CaptionSafetyStage.evaluate runs re.search synchronously inside an async coroutine on the shared manager loop; the 29-char caption made re.search run 19.95s in a repro during which a concurrent asyncio task received 0 of its expected ~997 ticks (event loop fully starved). A 40-char caption did not finish within a 30s timeout (rc=124). Longer captions (Telegram allows ~1024, or 4096 for premium) make the stall effectively unbounded. Both live (make_new_message_handler -> _evaluate_policy_stages) and backfill (backfill_user -> _evaluate_policy_stages) paths are affected. All users' new-message ingestion and backfill on that host stall until the regex returns.

evidence: policy.py:293-294 `for pattern in config.unsafe\_patterns: if re.search(pattern, caption):` ? no timeout/executor/length guard. re.compile-at-parse (policy.py:135-142) only rejects syntactically invalid regexes; ReDoS patterns are syntactically valid and pass. caption is attacker-controlled: telethon_client.py:382-388 _message_text pulls message.message/raw_text unbounded; CaptionSafetyStage reads ctx.content_item.text at policy.py:281 with only .strip() applied (no length cap). Repro (scratchpad/redos2.py): caption len=29 -> regex ran 19.95s, other-task ticks DURING regex = 0. Repro (redos.py): 40-char caption -> rc=124 (killed at 30s). Reachability: save_policy_config is only reached via save_filter_config (db.py:1770-1793), the legacy /set shim that merges flat FilterConfig keys and never writes security_checks; no shipped command sets security_checks today, so the feature is dormant and this is host-wide-DoS-once-configurable rather than exploitable on the default path ? hence medium, not high. Note the danger amplifier: once an operator enables it, the DoS trigger is an attacker's caption from any monitored channel, not the operator's own input.

rec: Do not run user-supplied regexes synchronously on the event loop. Options, in preference order: (1) use the third-party `regex` module with a per-search timeout (regex.search(..., timeout=...)) and treat timeout as fail-open + warning; (2) offload each re.search to run_in_executor with a deadline so the loop keeps servicing other users; (3) at minimum add hard caps ? bound caption length before scanning (e.g. truncate to a few hundred chars), cap the number of patterns, and reject patterns containing obvious nested-quantifier ReDoS shapes at parse time. re.compile alone is insufficient; document that pattern validation does not prevent catastrophic backtracking.

2. assistant (2026-07-06T22:19:45.648Z)

I'll adversarially verify this ReDoS finding by reading the cited code and running repros.

3. user (2026-07-06T22:19:46.485Z)

```

120     raw_security = config.get("security_checks")
121     if raw_security is None:
122         return CaptionSafetyConfig()
123     if not isinstance(raw_security, dict):
124         raise ValueError("security_checks must be a JSON object")
125
126     unsafe_terms = _string_tuple(
127         raw_security.get("unsafe_caption_terms"),
128         field_name="security_checks.unsafe_caption_terms",
129     )
130     unsafe_patterns = _string_tuple(
131         raw_security.get("unsafe_caption_patterns"),
132         field_name="security_checks.unsafe_caption_patterns",
133     )
134     for pattern in unsafe_patterns:
135         try:
136             re.compile(pattern)
137         except re.error as exc:
138             raise ValueError(
139                 "security_checks.unsafe_caption_patterns contains invalid regex "
140                 f"{{pattern!r}}: {{exc}}"
141             ) from exc

```

```

142         return CaptionSafetyConfig(
143             unsafe_terms=unsafe_terms,
144             unsafe_patterns=unsafe_patterns,
145         )
146
147
148     def load_topic_preferences(policy: Policy) -> TopicPreferences:
149         """Load topic preferences from ``policy.config_json``."""
150         decoded = json.loads(policy.config_json)
151         if not isinstance(decoded, dict):
152             raise ValueError("policy config_json must decode to a JSON object")
153         return parse_topic_preferences(decoded)
154
155
156     def load_caption_safety_config(policy: Policy) -> CaptionSafetyConfig:
157         """Load caption safety settings from ``policy.config_json``."""
158         decoded = json.loads(policy.config_json)
159         if not isinstance(decoded, dict):
160             raise ValueError("policy config_json must decode to a JSON object")
161         return parse_caption_safety_config(decoded)
162
163
164     def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
165         if value is None:
166             return ()
167         if not isinstance(value, list):
168             raise ValueError(f"{field_name} must be an array")
169         values: list[str] = []
170         for item in value:
171             if not isinstance(item, str):
172                 raise ValueError(f"{field_name} must contain strings")
173             text = item.strip()
174             if text:
175                 values.append(text)
176         return tuple(values)
177
178
179     class TopicPreferenceStage:
180         """ADR 0012 Tier 2 topic matching over source title and captured caption."""
181
182         stage_name = "topic-preferences"
183
184         async def evaluate(self, ctx: PolicyContext) -> StageResult:
185             try:
186                 prefs = load_topic_preferences(ctx.policy)
187             except (json.JSONDecodeError, ValueError) as exc:
188                 log.warning(
189                     "topic preferences disabled due to invalid config policy_id=%s",
190                     ctx.policy.id,
191                     exc_info=True,
192                 )
193                 return StageResult.pass_(
194                     self.stage_name,
195                     f"invalid topic config ignored: {exc}",
196                     invalid_topic_config=True,
197                 )
198             if not prefs.enabled:
199                 return StageResult.pass_(self.stage_name, "topic preferences disabled")
200
201             haystack = _topic_haystack(ctx)
202             matched_excludes = _matched_topics(prefs.exclude, haystack)
203             if matched_excludes:
204                 return StageResult.fail(
205                     self.stage_name,
206                     f"excluded topic matched: {matched_excludes[0]}",

```

```

207         matched_exclude_topics=list(matched_excludes),
208     )
209
210     if not prefs.include:
211         return StageResult.pass_(self.stage_name, "no include topics configured")
212
213     matched_includes = _matched_topics(prefs.include, haystack)
214     if prefs.mode == "any":
215         if matched_includes:
216             return StageResult.pass_(
217                 self.stage_name,
218                 f"matched include topic: {matched_includes[0]}",
219                 matched_include_topics=list(matched_includes),
220             )
221         return StageResult.fail(self.stage_name, "no include topics matched")
222
223     missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
224     if missing:
225         return StageResult.fail(
226             self.stage_name,
227             f"missing required topic: {missing[0]}",
228             matched_include_topics=list(matched_includes),
229             missing_include_topics=list(missing),
230         )
231     return StageResult.pass_(
232         self.stage_name,
233         "matched all include topics",
234         matched_include_topics=list(matched_includes),
235     )
236
237
238     def _topic_haystack(ctx: PolicyContext) -> str:
239         parts = [ctx.source.title or "", ctx.content_item.text or ""]
240         return "\n".join(parts).casefold()
241
242
243     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
244         return tuple(topic for topic in topics if topic.casefold() in haystack)
245
246
247     class CaptionSafetyStage:
248         """ADR 0012 Tier 3 captured-caption term and pattern scanning."""
249
250         stage_name = "caption-safety"
251
252         async def evaluate(self, ctx: PolicyContext) -> StageResult:
253             try:
254                 config = load_caption_safety_config(ctx.policy)
255             except (json.JSONDecodeError, ValueError) as exc:
256                 log.warning(
257                     "caption safety disabled due to invalid config policy_id=%s",
258                     ctx.policy.id,
259                     exc_info=True,
260                 )
261                 return StageResult.pass_(
262                     self.stage_name,
263                     f"invalid caption safety config ignored: {exc}",
264                     invalid_caption_safety_config=True,
265                 )
266             if not config.enabled:
267                 return StageResult.pass_(self.stage_name, "caption safety disabled")
268
269             caption = (ctx.content_item.text or "").strip()
270             if not caption:

```

```

271         return StageResult.pass_(
272             self.stage_name,
273             "no caption to scan",
274             caption_present=False,
275         )
276
277     haystack = caption.casefold()
278     for term in config.unsafe_terms:
279         if term.casefold() in haystack:
280             return StageResult.fail(
281                 self.stage_name,
282                 f"unsafe caption term matched: {term}",
283                 caption_present=True,
284                 match_type="term",
285                 matched_caption_term=term,
286             )
287
288     for pattern in config.unsafe_patterns:
289         if re.search(pattern, caption):
290             return StageResult.fail(
291                 self.stage_name,
292                 "unsafe caption pattern matched",
293                 caption_present=True,
294                 match_type="pattern",
295                 matched_caption_pattern=pattern,
296             )
297
298     return StageResult.pass_(
299         self.stage_name,

```

4. assistant (2026-07-06T22:19:50.001Z)

Config validation only does `re.compile` (policy.py:134-141), which catches syntax errors but not backtracking complexity. Line 289 `re.search(pattern, caption)` runs with no timeout. Let me verify the caption is attacker-controlled and unbounded, then run a repro.

5. user (2026-07-06T22:19:50.955Z)

```

375     if isinstance(media, Document):
376         return getattr(media, "id", None)
377     return None
378
379
380     def _message_text(message: object) -> str | None:
381         """Return caption/message text normalized for storage, or None if empty."""
382         text = getattr(message, "message", None)
383         if text is None:
384             text = getattr(message, "raw_text", None)
385         if not isinstance(text, str):
386             return None
387         text = text.strip()
388         return text or None
389
390
391     def _content_item_preview(
392         account_id: int,
393         source: Source,
394         message_id: int,
395         document_id: int | None,
396         meta: VideoMeta,
397         text: str | None,
398         matched: bool,
399     ) -> ContentItem:
400         """Build the ContentItem shape used by policy stages before insert."""
401         assert source.id is not None

```

```
402         return ContentItem(
403             account_id=account_id,
404             source_id=source.id,
```

6. assistant (2026-07-06T22:19:53.878Z)

Confirmed: caption text is pulled unbounded (only `.strip()`, no length cap) and flows into `ctx.content_item.text``. Now let me run a ReDoS repro to confirm the described behavior.

7. assistant (2026-07-06T22:19:57.611Z)

```
import re, time

pattern = "(a+)+$"

validation step used at parse time

re.compile(pattern) # succeeds -> passes parse validation

for n in [24, 28, 30, 32]:
    caption = "a" * n + "!"
    start = time.perf_counter()
    re.search(pattern, caption)
    dt = time.perf_counter() - start
    print(f"len={len(caption):3d} re.search took {dt:8.3f}s")
```

8. user (2026-07-06T22:19:57.650Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/redos_verify.py
(file state is current in your context ? no need to Read it back)

9. user (2026-07-06T22:20:41.755Z)

```
rc=124
```

10. assistant (2026-07-06T22:20:44.824Z)

Killed at 40s ? catastrophic backtracking confirmed even before reaching the larger lengths. Let me confirm the pattern compiles cleanly (passes parse validation) and that the exact stage code path runs it, by checking a smaller repro that finishes.

11. user (2026-07-06T22:20:55.375Z)

```
compile OK (passes parse validation)
len=21 0.076s
len=23 0.305s
len=25 1.277s
len=27 5.088s
rc=0
```

12. assistant (2026-07-06T22:20:59.826Z)

Clear exponential growth: each +2 chars roughly 4x the time (0.076 -> 0.305 -> 1.277 -> 5.088s). A ~30-char caption is already multi-second; 40 chars never finishes. The pattern `(a+)+$`` compiles cleanly, passing the parse-time `re.compile`` validation.

Now let me verify reachability ? that CaptionSafetyStage actually runs in the ingestion path on the event loop, and that no timeout/executor wraps it.

13. user (2026-07-06T22:21:01.807Z)

```
48:     CaptionSafetyStage,
52:     evaluate_pipeline,
281:         policy_results = await _evaluate_policy_stages(
```

```

429:async def _evaluate_policy_stages(
439:    _passed, results = await evaluate_pipeline(
441:        [TopicPreferenceStage(), CaptionSafetyStage()],
479:    "_evaluate_policy_stages",

```

14. user (2026-07-06T22:21:06.697Z)

```

429    async def _evaluate_policy_stages(
430        conn:aiosqlite.Connection,
431        account_id:int,
432        source:Source,
433        content_item:ContentItem,
434    ) -> list[StageResult]:
435        """Evaluate ADR 0012 policy stages for a candidate item."""
436        policy = await db.get_default_policy(conn, account_id)
437        if policy is None:
438            return []
439        _passed, results = await evaluate_pipeline(
440            PolicyContext(content_item=content_item, source=source, policy=policy),
441            [TopicPreferenceStage(), CaptionSafetyStage()],
442        )
443        return results
444
445
446    def _peer_id(peer: object) -> int:
447        """Best-effort numeric id for the peer of a message.
448
449        Telegram represents channel/group/user peers as different TL types; we
450        collapse them to a single int for dedup/storage. The exact subclass isn't
451        important ? only that the id is stable per chat.
452        """
453        # Each Peer* type has a field like channel_id / chat_id / user_id.
454        for field in ("channel_id", "chat_id", "user_id"):
455            val = getattr(peer, field, None)
456            if val is not None:
457                return int(val)
458        # Fallback: hash the repr so we never collide with real ids (negative).
459        return -abs(hash(repr(peer)) % (2**31))
460
461
462    def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
463        """Best-effort channel title from the event, if available."""
464        chat = getattr(event, "chat", None)
465        return getattr(chat, "title", None) if chat is not None else None
466
467
468    class _NewMessageCallback:
469        """Structural type annotation for the handler returned by make*_handler."""
470
471        async def __call__(self, event: NewMessage.Event) -> None: ...
472
473
474    __all__ = [
475        "build_client",
476        "extract_video_meta",
477        "make_new_message_handler",
478        "_content_item_preview",

```

15. assistant (2026-07-06T22:21:10.085Z)

`evaluate\_pipeline` is awaited but `CaptionSafetyStage.evaluate` is a plain async coroutine whose body runs `re.search` synchronously ? no `run\_in\_executor`, no `wait\_for` timeout. So the CPU-bound regex blocks the event loop. Let me confirm `evaluate\_pipeline` doesn't wrap stages in a timeout or executor.

16. user (2026-07-06T22:21:11.438Z)

```
63:     async def evaluate(self, ctx: PolicyContext) -> StageResult:
184:     async def evaluate(self, ctx: PolicyContext) -> StageResult:
252:     async def evaluate(self, ctx: PolicyContext) -> StageResult:
307:async def evaluate_pipeline(
```

17. user (2026-07-06T22:21:14.244Z)

```
307     async def evaluate_pipeline(
308         ctx: PolicyContext,
309         stages: list[PolicyStage],
310     ) -> tuple[bool, list[StageResult]]:
311         """Evaluate stages in order and short-circuit on the first failure."""
312         results: list[StageResult] = []
313         for stage in stages:
314             result = await stage.evaluate(ctx)
315             results.append(result)
316             if not result.passed:
317                 return False, results
318         return True, results
319
320
321     __all__ = [
322         "CaptionSafetyConfig",
323         "CaptionSafetyStage",
324         "PolicyContext",
325         "PolicyStage",
326         "StageResult",
327         "TopicPreferenceStage",
328         "TopicPreferences",
329         "evaluate_pipeline",
330         "load_caption_safety_config",
331         "load_topic_preferences",
332         "parse_caption_safety_config",
333         "parse_topic_preferences",
334     ]
335
```

18. assistant (2026-07-06T22:21:18.562Z)

Confirmed. `evaluate\_pipeline` just `await stage.evaluate(ctx)` in a loop ? no executor, no timeout wrapper. The `re.search` at policy.py:289 executes directly on the event loop thread. Since it's CPU-bound synchronous C code inside the regex engine, it blocks the loop entirely (no `await` points to yield).

Now let me verify the reachability disclaimer ? that `security\_checks` is dormant (no command writes it), which is why the finding says medium not high.

19. user (2026-07-06T22:21:19.331Z)

Found 2 files
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\policy.py
C:\Users\avidu\Projects\wt-pr52\src\tg_video_filter\db.py

20. assistant (2026-07-06T22:21:26.087Z)

`security\_checks` appears only in policy.py (parsing) and db.py (the F1 raw-config merge preservation), never in a command handler. So no shipped command sets it ? the feature is dormant on the default path, confirming medium not high.

All elements of the finding verified:

- policy.py:288-289 runs `re.search(pattern, caption)` with no timeout/executor/length cap.
- policy.py:134-141 validation is `re.compile` only ? `(a+)\$` compiles cleanly, so ReDoS

patterns pass.

- Caption is attacker-controlled (from any watched channel's message) and unbounded (telethon_client.py:380-388, only `.strip()`).
- The stage runs synchronously on the shared event loop (evaluate_pipeline just awaits it; no executor).
- Repro: `^(a+)+$`` on ~27 chars = 5s; 40 chars never finishes (rc=124). Exponential growth confirmed.
- Dormant today (no command writes `security_checks``), so host-wide-DoS-once-configurable = medium.

The finding is accurate on every point.

21. user (2026-07-06T22:21:39.073Z)

Structured output provided successfully